

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0704

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards

Assessment Tool Weightage: 40%

Annexure A Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

Aim: To generate a Star Topology where multiple hosts are connected to a central switch. The program displays the topology in ASCII format and shows the Cat6 UTP cable length for every host.

Algorithm

- Read the number of hosts.
- Create a central switch named SW1.
- Read the Cat6 cable length for each host.
- Connect every host to the central switch.
- Display the star topology.

- Display the host, switch, and cable length.
1. Verify that every host is connected to the switch.

Python Program

Star Topology Generator

```
def create_star_topology():
    n = int(input("Enter number of hosts: "))

    if n <= 0:
        print("Number of hosts must be greater than 0.")
        return

    switch = "SW1"
    hosts = []

    for i in range(1, n + 1):
        length = float(input(f"Enter Cat6 cable length for Host{i} (m):
"))

        if length <= 0:
            print("Cable length must be positive.")
            return

        hosts.append((f"Host{i}", length))

    print("\n--- STAR TOPOLOGY ---")
    print("      " + switch)

    for host, length in hosts:
        print("      |")
        print(f"      {host} -- {length} m")
```

```

print("\n--- CONNECTION DETAILS ---")
print("Host\tSwitch\tCable Length")

for host, length in hosts:
    print(f"{host}\t{switch}\t{length} m")

print("\nVerification:")
print(f"All {n} hosts are connected through {switch}.")
print("Star topology successfully created.")

```

```
create_star_topology()
```

Sample Output

```

Enter number of hosts: 4
Enter Cat6 cable length for Host1 (m): 10
Enter Cat6 cable length for Host2 (m): 15
Enter Cat6 cable length for Host3 (m): 20
Enter Cat6 cable length for Host4 (m): 12

```

```
--- STAR TOPOLOGY ---
```

```

      SW1
      |
Host1 -- 10.0 m
      |
Host2 -- 15.0 m
      |
Host3 -- 20.0 m
      |
Host4 -- 12.0 m

```

```
--- CONNECTION DETAILS ---
```

```
Host  Switch  Cable Length
```

Host1	SW1	10.0 m
Host2	SW1	15.0 m
Host3	SW1	20.0 m
Host4	SW1	12.0 m

Verification:

All 4 hosts are connected through SW1.

Star topology successfully created.

Explanation: In a star topology, every host has a separate connection to a central switch. If one cable fails, only that host is affected. The other hosts can continue communicating, which makes the topology reliable and easy to manage.

2. Write a Python function named `validate_segment(length_m)` to verify whether a given UTP cable segment satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of 100 meters. The function should return `True` for valid cable lengths and return `False` along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least three different test cases and interpret the results.

Aim: To verify whether a UTP cable segment satisfies the 100-meter maximum length specified for Ethernet copper cabling.

Algorithm

- Define the maximum length as 100 meters.
- Create `validate_segment(length_m)`.
- Check whether the length is between 0 and 100 meters.
- Return `True` for a valid length.
- Return `False` and an error message for an invalid length.

- Test the function with at least three values.

Python Program

```
# UTP Cable Segment Validator

MAX_LENGTH = 100

def validate_segment(length_m):
    if length_m <= 0:
        return False, "Error: Cable length must be greater than 0 meters."

    if length_m > MAX_LENGTH:
        return False, "Error: Cable length exceeds the 100-meter limit."

    return True, "Valid UTP cable segment."

# Test Case 1
length = 50
result, message = validate_segment(length)
print(f"Test 1: {length} m -> {result}, {message}")

# Test Case 2
length = 100
result, message = validate_segment(length)
print(f"Test 2: {length} m -> {result}, {message}")

# Test Case 3
length = 120
result, message = validate_segment(length)
print(f"Test 3: {length} m -> {result}, {message}")
```

```
# Test Case 4
length = -10
result, message = validate_segment(length)
print(f'Test 4: {length} m -> {result}, {message}')
```

Sample Output

Test 1: 50 m -> True, Valid UTP cable segment.
Test 2: 100 m -> True, Valid UTP cable segment.
Test 3: 120 m -> False, Error: Cable length exceeds the 100-meter limit.
Test 4: -10 m -> False, Error: Cable length must be greater than 0 meters.

Test Case Interpretation

Test Case	Length	Result
1	50 m	Valid
2	100 m	Valid
3	120 m	Invalid
4	-10 m	Invalid

Conclusion: Cable lengths up to 100 meters are accepted, while lengths greater than 100 meters are rejected.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named csma_log.txt, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

Aim: To simulate Carrier Sense Multiple Access with Collision Detection (CSMA/CD) for four hosts connected to a central switch.

Algorithm

- Create four hosts.
- Randomly select hosts that want to transmit.
- Check whether more than one host transmits simultaneously.
- If only one host transmits, mark it successful.
- If multiple hosts transmit, record a collision.
- Generate a random back-off value.
- Retry transmission after the back-off.
- Store collision information in a log file.
- Continue until all frames are transmitted.

Python Program

```
# CSMA/CD Simulation
```

```
import random
import time
```

```
HOSTS = ["Host1", "Host2", "Host3", "Host4"]
FRAMES = {
    "Host1": 2,
    "Host2": 2,
    "Host3": 2,
    "Host4": 2
}
```

```
log_file = "csma_log.txt"
```

```
def log(message):
    with open(log_file, "a") as file:
        file.write(message + "\n")
```

```

def csma_cd():
    pending = []

    for host in HOSTS:
        for frame in range(1, FRAMES[host] + 1):
            pending.append((host, frame))

    attempt = 1
    open(log_file, "w").close()

    while pending:
        print(f"\nTransmission Attempt {attempt}")

        transmit_count = random.randint(1, min(3, len(pending)))
        selected = random.sample(pending, transmit_count)
        transmitters = [item[0] for item in selected]

        print("Transmitting:", ", ".join(transmitters))

        if len(selected) > 1:
            message = (
                f"Collision detected between: "
                f"{', '.join(transmitters)}"
            )

            print(message)
            log(message)

            backoff = random.randint(1, 5)
            print(f"Back-off interval: {backoff}")
            log(f"Back-off interval: {backoff}")

```



```

        time.sleep(0.2)

    else:
        host, frame = selected[0]

        print(f'{host} Frame {frame} transmitted successfully.')
        log(f'{host} Frame {frame} transmitted successfully.')

        pending.remove(selected[0])

    attempt += 1

    print("\nAll frames transmitted successfully.")
    print(f'Collision log saved in {log_file}')

csma_cd()

```

Sample Output

```

Transmission Attempt 1
Transmitting: Host2, Host4
Collision detected between: Host2, Host4
Back-off interval: 3

```

```

Transmission Attempt 2
Transmitting: Host1
Host1 Frame 1 transmitted successfully.

```

```

Transmission Attempt 3
Transmitting: Host3, Host4
Collision detected between: Host3, Host4
Back-off interval: 2

```

Transmission Attempt 4
Transmitting: Host2
Host2 Frame 1 transmitted successfully.

...

All frames transmitted successfully.
Collision log saved in csma_log.txt

Explanation: CSMA/CD allows devices to sense the communication medium before transmission. If two or more devices transmit at the same time, a collision occurs. The devices wait for a randomly selected back-off period before trying again.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

Aim: To simulate communication between two hosts through a central switch using Python's socket library. The server acts as the central switch, receives frames, learns MAC addresses, maintains a MAC address table, and forwards frames to the appropriate destination.

Server Program – server.py

```
import socket
```

```
HOST = "127.0.0.1"
```

```
PORT = 5000
```

```
mac_table = {}
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(5)
```

```
print("Central Switch Server Started...")
print("Waiting for clients...")
```

```
clients = {}
```

```
while len(clients) < 2:
    conn, addr = server.accept()
```

```
    hello = conn.recv(1024).decode()
    mac, host_name = hello.split(",")
```

```
    clients[host_name] = conn
    mac_table[mac] = host_name
```

```
    print(f" {host_name} connected with MAC {mac}")
```

```
print("\nMAC Address Table")
print("-----")
```

```
for mac, port in mac_table.items():
    print(f" {mac} -> {port}")
```

```
print("\nSwitch is ready for communication.")
```

```

while True:
    for host, conn in list(clients.items()):
        try:
            conn.settimeout(0.1)
            data = conn.recv(1024)

            if not data:
                continue

            message = data.decode()
            source, destination, frame = message.split("|")

            print(f"\nFrame received from {source}")
            print(f"Destination: {destination}")
            print(f>Data: {frame}")

            mac_table[source] = host

            print("\nUpdated MAC Address Table")
            for mac, port in mac_table.items():
                print(f"{mac} -> {port}")

            if destination in mac_table:
                destination_host = mac_table[destination]

                clients[destination_host].send(
                    f"From {source}: {frame}".encode()
                )

                print(f"Frame forwarded to {destination_host}")

        except socket.timeout:

```

```
pass
```

Client Program – client.py

```
import socket
```

```
import threading
```

```
HOST = "127.0.0.1"
```

```
PORT = 5000
```

```
host_name = input("Enter host name: ")
```

```
mac_address = input("Enter MAC address: ")
```

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
client.connect((HOST, PORT))
```

```
client.send(f'{mac_address},{host_name}'.encode())
```

```
def receive_data():
```

```
    while True:
```

```
        try:
```

```
            data = client.recv(1024).decode()
```

```
            if data:
```

```
                print("\nReceived:", data)
```

```
        except:
```

```
            break
```

```
thread = threading.Thread(
```

```
    target=receive_data,
```

```
    daemon=True
```

```
)
```

```
thread.start()
```

```
while True:
```

```
    destination = input("\nEnter destination MAC: ")
```

```
    message = input("Enter frame data: ")
```

```
    frame = f"{mac_address}|{destination}|{message}"
```

```
    client.send(frame.encode())
```

Execution

Terminal 1:

```
python server.py
```

Terminal 2:

```
python client.py
```

Terminal 3:

```
python client.py
```

Example Output

```
Enter host name: Host1
```

```
Enter MAC address: AA:BB:CC:DD:EE:01
```

```
Enter destination MAC: AA:BB:CC:DD:EE:02
```

```
Enter frame data: Hello Host2
```

```
Host2:
```

```
Received: From AA:BB:CC:DD:EE:01: Hello Host2
```

Explanation: The switch uses MAC learning to associate a source MAC address with a port/host. When a frame arrives, the source MAC is learned, the destination MAC is searched in the MAC table, and the frame is forwarded to the corresponding port if the destination is known.

5. Design and implement a Python class named Star Switch that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port (host_id)` to connect a host, `remove_port (host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

Aim: To create a Python class that models the basic functionality of an Ethernet switch. The class supports adding ports, removing ports, broadcasting frames, maintaining active ports, and logging broadcast operations.

Algorithm

2. Create a StarSwitch class.
3. Initialize an empty list of active ports.
4. Add hosts using `add_port()`.
5. Remove hosts using `remove_port()`.
6. Use `broadcast()` to transmit frames.
7. Exclude the source host.
8. Display the successful destinations.
9. Store broadcast operations in a log.

Python Program

```
# StarSwitch Ethernet Switch Simulation
```

```
class StarSwitch:
```

```

def __init__(self):
    self.active_ports = []
    self.log = []

def add_port(self, host_id):
    if host_id not in self.active_ports:
        self.active_ports.append(host_id)
        print(f'{host_id} connected successfully.')
    else:
        print(f'{host_id} is already connected.')

def remove_port(self, host_id):
    if host_id in self.active_ports:
        self.active_ports.remove(host_id)
        print(f'{host_id} disconnected successfully.')
    else:
        print(f'{host_id} is not connected.')

def broadcast(self, frame):
    source = frame["source"]
    data = frame["data"]

    destinations = []

    for host in self.active_ports:
        if host != source:
            destinations.append(host)

    message = (
        f'Broadcast from {source}: '
        f'{data} -> {destinations}'
    )

```



```

self.log.append(message)

print("\nBroadcast Operation")
print("-----")
print(f"Source: {source}")
print(f"Frame: {data}")

if destinations:
    print("Frame successfully received by:")

    for host in destinations:
        print(f"- {host}")
    else:
        print("No destination hosts available.")

def show_ports(self):
    print("\nActive Ports:")
    if self.active_ports:
        for host in self.active_ports:
            print(f"- {host}")
    else:
        print("No active ports.")

def show_log(self):
    print("\nBroadcast Log:")
    for entry in self.log:
        print(entry)

switch = StarSwitch()

```

```
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")
```

```
switch.show_ports()
```

```
frame1 = {
    "source": "Host1",
    "data": "Hello Ethernet"
}
```

```
switch.broadcast(frame1)
```

```
switch.remove_port("Host3")
switch.show_ports()
```

```
frame2 = {
    "source": "Host2",
    "data": "Network Message"
}
```

```
switch.broadcast(frame2)
switch.show_log()
```

Sample Output

```
Host1 connected successfully.
Host2 connected successfully.
Host3 connected successfully.
Host4 connected successfully.
```

Active Ports:

- Host1

- Host2
- Host3
- Host4

Broadcast Operation

Source: Host1

Frame: Hello Ethernet

Frame successfully received by:

- Host2
- Host3
- Host4

Host3 disconnected successfully.

Active Ports:

- Host1
- Host2
- Host4

Broadcast Operation

Source: Host2

Frame: Network Message

Frame successfully received by:

- Host1
- Host4

Explanation: Ethernet broadcasting allows a frame to be transmitted to all connected hosts except the source host. It is useful when the destination MAC address is unknown or when communication is intended for multiple hosts.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done